IMPLEMENTATIONGUIDE

xOS Experimentation Brain

Building on Claude Code

Architecture, Folder Structure, Skills, Agents & MCPs

Prepared for Bjarn Brunenberg

GitHub: bjarnbrunenberg-cloud/xOS-Brain

February 2026

CONFIDENTIAL

Executive Overview

This document is your **complete technical blueprint** for building the xOS Experimentation Brain as a Claude Code project. It covers every layer of the architecture: from the **CLAUDE.md** master configuration file, through the folder structure for skills, agents, and MCPs, all the way to the step-by-step build order that gets you from an empty GitHub repository to a fully operational experimentation OS.

The xOS Brain is built on top of **Claude Code**, Anthropic's agentic coding tool. Claude Code provides a powerful framework with three core extension mechanisms that map perfectly to the xOS architecture:

Component	Claude Code Feature	xOS Brain Purpose
Skills	Markdown instruction files that activate based on context	Each of the 9 xOS modules becomes a skill with specialized prompts and workflows
Agents	Sub-agents with scoped tools, models, and persistent memory	Autonomous workers that handle complex multi-step experiment tasks
MCPs	Model Context Protocol servers for external tool integration	Connect to VWO, Airtable, Google Sheets, analytics platforms, and APIs

> **Key Principle: Two-Layer Architecture**
>
> The Operating Engine (your skills, agents, and MCPs) is the secret sauce. It lives in .claude/ and .mcp.json.
>
> The Context Layer (client-specific data, templates, knowledge) lives in contexts/ and is swappable per engagement.
>
> This separation means you build the engine once and reuse it across every client.

CLAUDE.md — The Master Configuration

The **CLAUDE.md** file is the single most important file in your xOS Brain repository. Claude Code automatically reads this file at the start of every session, making it your project's **persistent memory and instruction set**. Think of it as the brain's DNA — it tells Claude who it is, what it can do, and how it should behave.

What Goes in CLAUDE.md

Your CLAUDE.md should contain the following sections, structured in this exact order:

Section	Purpose	Example Content
Project Identity	Tells Claude what this project is and its role	"You are xOS Brain, an AI experimentation operating system..."
Architecture Overview	Documents the two-layer architecture and key concepts	Operating Engine vs. Context Layer, the 9 modules
Coding Conventions	Standards for code style, naming, and patterns	TypeScript preferred, camelCase, error handling patterns
Skill Index	Reference list of all available skills and their triggers	"Use @hypothesis-factory when user needs to generate hypotheses"
Agent Registry	Documents all agents, their capabilities, and when to use them	"experiment-designer agent handles multi-step test creation"
MCP Configuration	Lists all active MCP servers and their capabilities	"VWO MCP for A/B test management, Airtable for experiment tracking"
Workflow Patterns	Common multi-step workflows and how they chain	"Hypothesis Design Launch Monitor Analyze"
Safety & Guardrails	Rules Claude must always follow	"Never delete running experiments. Always confirm before launching."

Complete CLAUDE.md Template

Below is the complete CLAUDE.md file you should place at the root of your xOS-Brain repository. Copy this exactly and customize the placeholders.

```
# xOS Experimentation Brain

## Identity
You are **xOS Brain**, an AI-powered experimentation operating system
built on Claude Code. Your purpose is to help product teams design,
```

```
run, analyze, and learn from business experiments systematically.

## Architecture
xOS Brain uses a two-layer architecture:
- **Operating Engine** (.claude/, .mcp.json): Skills, agents, MCPs
- **Context Layer** (contexts/): Client data, templates, knowledge base

## Core Modules (Skills)
1. Strategic Alignment Engine (@strategic-alignment)
2. Hypothesis Factory (@hypothesis-factory)
3. Experiment Designer (@experiment-designer)
4. Experiment Monitor (@experiment-monitor)
5. Analysis Engine (@analysis-engine)
6. Report Generator (@report-generator)
7. Learning Repository (@learning-repository)
8. Next Best Test Engine (@next-best-test)
9. xOS Dashboard (@dashboard)

## Agent Registry
- experiment-orchestrator: Full experiment lifecycle management
- data-analyst: Statistical analysis and data interpretation
- report-writer: Generating professional experiment reports
- monitor-agent: Tracking running experiments for anomalies

## MCP Servers
All MCP servers are configured in .mcp.json at project root.
- VWO: A/B test creation and management
- Airtable: Experiment tracking database
- Google Sheets: Data import/export and reporting
- Analytics: GA4/Mixpanel/Amplitude data fetching

## Workflow Chains
Standard experiment lifecycle:
Strategic Alignment → Hypothesis → Design → Launch → Monitor → Analyze → Report → Learn

## Safety Rules
- NEVER delete or stop a running experiment without explicit user confirmation
- ALWAYS validate statistical significance before declaring winners
- ALWAYS save experiment results to the Learning Repository
- CONFIRM with user before launching any experiment to production
- LOG all major decisions and rationale to the experiment record
```

Pro Tip: CLAUDE.md Layering

Claude Code supports CLAUDE.md files at multiple levels. Place the master file at the repository root, and add specialized CLAUDE.md files inside subdirectories for context-specific instructions.

Example: contexts/acme-corp/CLAUDE.md can contain ACME-specific business rules, brand voice, and KPI targets.

Complete Folder Structure

Below is the **complete, definitive folder structure** for the xOS-Brain GitHub repository. Every file and folder is intentional. This structure follows Claude Code conventions while implementing the xOS two-layer architecture.

```
xOS-Brain/
■■■ CLAUDE.md # Master project memory & instructions
■■■ .mcp.json # MCP server configurations (project scope)
■■■ .gitignore # Git ignore rules
■■■ README.md # Public repository documentation
■■■ package.json # Node.js dependencies (MCP servers, tools)
■
■■■ .claude/ # Claude Code configuration directory
■ ■■■ settings.json # Project-wide Claude Code settings
■ ■■■ settings.local.json # Local settings (gitignored)
■ ■
■ ■■■ skills/ # ■ SKILLS DIRECTORY
■ ■ ■■■ strategic-alignment.md # Module 1: Strategic Alignment Engine
■ ■ ■■■ hypothesis-factory.md # Module 2: Hypothesis Factory
■ ■ ■■■ experiment-designer.md # Module 3: Experiment Designer
■ ■ ■■■ experiment-monitor.md # Module 4: Experiment Monitor
■ ■ ■■■ analysis-engine.md # Module 5: Analysis Engine
■ ■ ■■■ report-generator.md # Module 6: Report Generator
■ ■ ■■■ learning-repository.md # Module 7: Learning Repository
■ ■ ■■■ next-best-test.md # Module 8: Next Best Test Engine
■ ■ ■■■ dashboard.md # Module 9: xOS Dashboard
■ ■
■ ■■■ agents/ # ■ AGENTS DIRECTORY
■ ■ ■■■ experiment-orchestrator.md # Full lifecycle experiment agent
■ ■ ■■■ data-analyst.md # Statistical analysis agent
■ ■ ■■■ report-writer.md # Report generation agent
■ ■ ■■■ monitor-agent.md # Experiment monitoring agent
■ ■
■ ■■■ agent-memory/ # Persistent agent memory storage
■ ■■■ experiment-orchestrator/ # Orchestrator's learned patterns
■ ■■■ data-analyst/ # Analyst's learned patterns
■ ■■■ monitor-agent/ # Monitor's learned patterns
■
■■■ contexts/ # ■ CONTEXT LAYER (client-specific)
■ ■■■ _template/ # Template for new client contexts
■ ■ ■■■ CLAUDE.md # Client-specific instructions template
■ ■ ■■■ config.yaml # Client config (KPIs, goals, tools)
■ ■ ■■■ brand-voice.md # Client's brand voice & terminology
■ ■ ■■■ knowledge/ # Client-specific knowledge base
■ ■ ■■■ past-experiments.md # Historical experiment data
■ ■ ■■■ business-model.md # Business model & value props
```

```
■ ■ ■■■ constraints.md # Technical/business constraints
■ ■
■ ■■■ acme-corp/ # Example: ACME Corp context
■ ■■■ CLAUDE.md # ACME-specific instructions
■ ■■■ config.yaml # ACME config
■ ■■■ brand-voice.md # ACME brand voice
■ ■■■ knowledge/ # ACME knowledge base
■
■■■ templates/ # Reusable experiment templates
■ ■■■ hypotheses/ # Hypothesis templates by type
■ ■ ■■■ conversion-hypothesis.md
■ ■ ■■■ retention-hypothesis.md
■ ■ ■■■ pricing-hypothesis.md
■ ■■■ experiments/ # Experiment design templates
■ ■ ■■■ ab-test.md
■ ■ ■■■ fake-door.md
■ ■ ■■■ landing-page.md
■ ■ ■■■ concierge-mvp.md
■ ■■■ reports/ # Report templates
■ ■■■ experiment-report.md
■ ■■■ weekly-summary.md
■
■■■ mcp-servers/ # ■ Custom MCP server implementations
■ ■■■ vwo/ # VWO A/B testing MCP server
■ ■ ■■■ index.js
■ ■ ■■■ package.json
■ ■■■ airtable/ # Airtable experiment tracker MCP
■ ■ ■■■ index.js
■ ■ ■■■ package.json
■ ■■■ analytics/ # Analytics data fetcher MCP
■ ■■■ index.js
■ ■■■ package.json
■
■■■ docs/ # Documentation & guides
■■■ architecture.md # Architecture deep-dive
■■■ getting-started.md # Onboarding guide
■■■ mcp-development.md # How to build custom MCPs
```

Folder Purpose Reference

Folder	Scope	Git Status	Purpose
.claude/skills/	Project	Committed	Skill markdown files — one per xOS module. Auto-triggered by Claude.

Folder	Scope	Git Status	Purpose
.claude/agents/	Project	Committed	Sub-agent definitions — autonomous workers with scoped tools and memory.
.claude/agent-memory/	Project	Committed	Persistent memory files that agents accumulate over time.
.claude/settings.json	Project	Committed	Shared project settings: permission rules, allowed tools.
.claude/settings.local.json	Local	Gitignored	Personal settings: API keys, local paths, model preferences.
.mcp.json	Project	Committed	MCP server configurations with env var placeholders for secrets.
contexts/	Project	Committed	Client-specific data, configs, and knowledge bases.
templates/	Project	Committed	Reusable templates for hypotheses, experiments, and reports.
mcp-servers/	Project	Committed	Custom MCP server source code for xOS integrations.
docs/	Project	Committed	Architecture docs, guides, and MCP development tutorials.

Skills — The 9 xOS Modules

Skills are the heart of xOS Brain. Each of the **9 core modules** from the Product Vision becomes a **skill file** in the .claude/skills/ directory. Skills are markdown files that Claude Code automatically loads when the conversation matches the skill's description.

How Skills Work in Claude Code

A skill file has two parts:

- **Description** (the first paragraph) — Claude uses this to decide when to activate the skill. Think of it as the trigger condition.
- **Instructions** (everything after) — The detailed prompt that tells Claude exactly how to perform the task when the skill activates.

> **When to Use Skills vs. Agents**
>
> Use SKILLS when: The task is conversational, needs human-in-the-loop guidance, or involves a single-step workflow. Skills run in the main conversation with the user.
>
> Use AGENTS when: The task requires autonomous multi-step execution, needs different tool permissions, or should run in the background with persistent memory.
>
> Example: "Generate 5 hypotheses" = Skill (user reviews each one). "Run the full experiment lifecycle" = Agent (autonomous multi-step).

Skill File Structure

Every skill follows this exact markdown structure:

```
---
description: A concise description of when this skill should activate.
Claude uses this text to match user intent to the right skill.
---

# Skill Name

## Context
[Background information Claude needs to execute this skill properly]

## Instructions
[Step-by-step instructions for Claude to follow]

## Output Format
[Expected deliverable format and structure]

## Examples
[Concrete examples of good outputs]
```

```
## Guardrails
[Rules and constraints Claude must respect]
```

Complete Skill Reference

Below is the complete reference for all 9 xOS skills with their descriptions, key responsibilities, and the MCP tools each skill requires access to.

#	Skill File	Description Trigger	Key Tools/MCPs Used
1	**strategic-alignment.md**	Activate when the user wants to evaluate strategic fit, check OKR alignment, or score an experiment idea against business goals	Airtable (read OKRs), contexts/ (client strategy)
2	**hypothesis-factory.md**	Activate when the user wants to generate, refine, or score experiment hypotheses using structured frameworks	templates/hypotheses/, contexts/ (past experiments)
3	**experiment-designer.md**	Activate when the user wants to design an experiment, select a test type, calculate sample size, or create a test plan	VWO MCP (test setup), templates/experiments/
4	**experiment-monitor.md**	Activate when the user wants to check experiment progress, detect anomalies, or review real-time metrics	VWO MCP (live data), Analytics MCP (metrics)
5	**analysis-engine.md**	Activate when the user wants to analyze experiment results, calculate statistical significance, or interpret data	Analytics MCP, VWO MCP (results data)
6	**report-generator.md**	Activate when the user wants to create an experiment report, weekly summary, or executive presentation	Airtable (experiment records), templates/reports/
7	**learning-repository.md**	Activate when the user wants to search past learnings, add an insight, or find patterns across experiments	Airtable (learnings DB), agent-memory/

#	Skill File	Description Trigger	Key Tools/MCPs Used
8	**next-best-test.md**	Activate when the user wants recommendations for what to test next based on past results and strategic priorities	Airtable (experiment history), contexts/ (strategy)
9	**dashboard.md**	Activate when the user wants to see experiment portfolio status, pipeline overview, or key metrics summary	VWO MCP, Airtable, Analytics MCP

Example: Hypothesis Factory Skill

Here is the complete skill file for the Hypothesis Factory module. Use this as a reference pattern when building the other 8 skills.

```
---
description: Activate when the user wants to generate, refine, or score
experiment hypotheses using structured frameworks. Triggers on keywords
like 'hypothesis', 'idea', 'assumption', 'what if', 'test idea',
'experiment idea', or 'generate hypotheses'.
---

# Hypothesis Factory

## Context
You are the Hypothesis Factory module of xOS Brain. Your job is to help
teams generate high-quality, testable experiment hypotheses using proven
frameworks. Every hypothesis must follow the structured format and be
scored for potential impact, confidence, and ease of testing (ICE).

## Hypothesis Format
Every hypothesis MUST follow this structure:
- **If** [specific change/intervention]
- **Then** [expected measurable outcome]
- **Because** [underlying assumption/rationale]

## Instructions
1. Ask the user for the business context:
- What area are they focused on? (conversion, retention, revenue, etc.)
- What data or observations triggered this exploration?
- Are there any constraints? (timeline, budget, technical)

2. Load the relevant client context:
```

```
- Read the client's CLAUDE.md for business goals
- Check contexts/{client}/knowledge/past-experiments.md for learnings
- Review contexts/{client}/config.yaml for current KPIs

3. Generate 5-7 hypotheses using the IF/THEN/BECAUSE format

4. Score each hypothesis using the ICE framework:
- **Impact** (1-10): How big will the effect be if we're right?
- **Confidence** (1-10): How confident are we this will work?
- **Ease** (1-10): How easy is this to test?
- **ICE Score** = (Impact + Confidence + Ease) / 3

5. Rank hypotheses by ICE score and recommend the top 3

6. For each recommended hypothesis, suggest:
- The best experiment type (A/B test, fake door, survey, etc.)
- Key metric to track
- Estimated test duration

## Output Format
Present results as a structured table with columns:
| # | Hypothesis (If/Then/Because) | Impact | Confidence | Ease | ICE | Recommended Test |

## Guardrails
- Never generate fewer than 3 hypotheses
- Always include the BECAUSE clause (it captures the assumption)
- Flag any hypothesis that conflicts with active experiments
- Check the Learning Repository for similar past tests before recommending
```

Example: Experiment Designer Skill

Here is the Experiment Designer skill. Notice how it references specific MCP tools and templates.

```
---
description: Activate when the user wants to design an experiment, select
a test type, calculate sample size, or create a detailed test plan.
Triggers on 'design experiment', 'test plan', 'sample size', 'A/B test',
'create experiment', 'set up test'.
---

# Experiment Designer

## Context
You are the Experiment Designer module of xOS Brain. Your role is to
transform validated hypotheses into rigorous, actionable experiment plans.
```

```
You use David Bland's Testing Business Ideas framework for test type
selection and ensure statistical rigor in all quantitative tests.

## Test Type Selection Matrix
Based on the hypothesis type and available resources, recommend from:
- **Discovery**: Customer interviews, surveys, search trend analysis
- **Validation**: Fake door tests, landing page tests, concierge MVP
- **Optimization**: A/B tests, multivariate tests, bandit algorithms

## Instructions
1. Receive the hypothesis (from Hypothesis Factory or user input)
2. Classify the assumption type: Desirability / Viability / Feasibility
3. Recommend the appropriate test type from the matrix above
4. Load the corresponding template from templates/experiments/
5. Calculate required sample size (for quantitative tests):
- Baseline conversion rate
- Minimum detectable effect (MDE)
- Statistical significance level (default: 95%)
- Statistical power (default: 80%)
6. Design the full experiment plan:
- Hypothesis statement
- Test type and rationale
- Primary and secondary metrics
- Sample size and estimated duration
- Audience targeting criteria
- Success/failure criteria
- Risk assessment and mitigation
7. If user approves, use VWO MCP to set up the test
8. Record the experiment in Airtable via MCP

## Guardrails
- ALWAYS calculate sample size before recommending test duration
- NEVER recommend launching without defined success criteria
- WARN if estimated duration exceeds 4 weeks (risk of external factors)
- CHECK for conflicts with other running experiments on same audience
```

Agents — Autonomous Workers

While skills operate **within the main conversation** (meaning you interact with them directly), agents are **autonomous sub-agents** that can be dispatched to handle complex, multi-step tasks independently. They have their own tool permissions, can use specific models, and maintain persistent memory across sessions.

How Agents Work in Claude Code

Each agent is defined as a markdown file in .claude/agents/ with a YAML frontmatter header that specifies:

- **name** — How you reference the agent (e.g., "data-analyst")
- **description** — What the agent does (used for selection matching)
- **model** — Which Claude model to use (opus for complex reasoning, sonnet for speed, haiku for simple tasks)
- **tools** — Which tools the agent can access (restrict to only what it needs)
- **skills** — Which skills to preload into the agent's context
- **memory** — Persistent memory scope (project-level or user-level)

Agent File Structure

```
---
name: agent-name
description: What this agent does and when to use it.
model: sonnet # Options: opus, sonnet, haiku
allowedTools:
- Read
- Write
- Edit
- Bash
- mcp__vwo__* # All VWO MCP tools
- mcp__airtable__*
skills:
- experiment-designer
- analysis-engine
memory:
scope: project # Options: project, user, local
---

# Agent System Prompt
[Detailed instructions for the agent's behavior]
```

xOS Agent Registry

Agent	Model	Purpose	Skills Preloaded	Key MCP Access
experiment-orchestrator	opus	Manages full experiment lifecycle from hypothesis to learnings	All 9 skills	VWO, Airtable, Analytics
data-analyst	sonnet	Performs statistical analysis, significance testing, and data interpretation	analysis-engine	Analytics, VWO (results)
report-writer	sonnet	Generates professional experiment reports and executive summaries	report-generator	Airtable (records)
monitor-agent	haiku	Lightweight agent that checks running experiments for anomalies and alerts	experiment-monitor	VWO (live data), Analytics

Example: Experiment Orchestrator Agent

This is the most important agent — it manages the full experiment lifecycle autonomously.

```
---
name: experiment-orchestrator
description: Manages the full experiment lifecycle from hypothesis creation
through design, launch, monitoring, analysis, and learning extraction.
Use this agent when you want to run a complete experiment end-to-end.
model: opus
allowedTools:
- Read
- Write
- Edit
- Bash
- Glob
- Grep
- mcp__vwo__*
- mcp__airtable__*
```

```
- mcp__analytics__*
skills:
- strategic-alignment
- hypothesis-factory
- experiment-designer
- experiment-monitor
- analysis-engine
- report-generator
- learning-repository
memory:
scope: project
---

# Experiment Orchestrator

You are the Experiment Orchestrator for xOS Brain. Your role is to manage
complete experiment lifecycles autonomously while keeping the user informed
at key decision points.

## Workflow
1. Receive experiment brief from user
2. Check strategic alignment (use @strategic-alignment skill)
3. Generate and score hypotheses (use @hypothesis-factory skill)
4. Present top hypotheses to user for selection
5. Design the experiment (use @experiment-designer skill)
6. Present experiment plan to user for approval
7. On approval, launch via VWO MCP
8. Monitor progress (use @experiment-monitor skill)
9. When complete, run analysis (use @analysis-engine skill)
10. Generate report (use @report-generator skill)
11. Extract and store learnings (use @learning-repository skill)

## Decision Points (ALWAYS pause for user input)
- After hypothesis scoring: "Here are the top hypotheses. Which to test?"
- After experiment design: "Here's the plan. Approve to launch?"
- After analysis: "Here are the results. Any questions before I file?"

## Memory
After each completed experiment, update your memory with:
- What worked well in this experiment cycle
- Any process improvements identified
- Patterns observed across experiments
```

Example: Data Analyst Agent

```
---
```

```
name: data-analyst
description: Performs statistical analysis on experiment data. Calculates
significance, confidence intervals, and effect sizes. Use when experiment
results need rigorous statistical interpretation.
model: sonnet
allowedTools:
- Read
- Bash # For running Python statistical scripts
- Write
- mcp__analytics__*
- mcp__vwo__get_results
- mcp__vwo__get_experiment_data
skills:
- analysis-engine
memory:
scope: project
---

# Data Analyst Agent

You are the Data Analyst for xOS Brain. You specialize in statistical
analysis of experiment data with a focus on:

## Core Capabilities
- Frequentist significance testing (z-test, t-test, chi-squared)
- Bayesian analysis with posterior probability estimation
- Sample Ratio Mismatch (SRM) detection
- Segmented analysis (by device, geography, user segment)
- Confidence interval calculation
- Effect size estimation (Cohen's d, relative lift)

## Rules
- ALWAYS check for SRM before interpreting results
- NEVER declare a winner at p > 0.05 without explicit user consent
- ALWAYS report confidence intervals alongside point estimates
- FLAG results where the confidence interval crosses zero
- USE Python (scipy.stats) for calculations, not mental math
```

MCPs — External Tool Integration

The **Model Context Protocol (MCP)** is how xOS Brain connects to external tools and services. MCPs give Claude the ability to read from and write to platforms like VWO, Airtable, Google Sheets, and analytics tools. The MCP configuration lives in a single file: **.mcp.json** at the project root.

How MCP Works

An MCP server is a lightweight process that exposes tools to Claude Code. Each server provides a set of functions that Claude can call. For example, a VWO MCP server might expose tools like create_experiment, get_results, and pause_test.

There are three types of MCP servers you will use:

- **Community/third-party MCPs** — Pre-built servers from npm packages (e.g., @airtable/mcp-server)
- **Custom MCPs** — Servers you build yourself in mcp-servers/ (e.g., your VWO integration)
- **HTTP MCPs** — Remote servers accessed via URL (e.g., a hosted analytics API)

The .mcp.json File

This file defines all MCP servers for the project. It is committed to Git so the whole team shares the same tool configuration. Secrets are passed via environment variables, never hardcoded.

```
{
"mcpServers": {
"vwo": {
"type": "stdio",
"command": "node",
"args": ["mcp-servers/vwo/index.js"],
"env": {
"VWO_API_TOKEN": "${VWO_API_TOKEN}",
"VWO_ACCOUNT_ID": "${VWO_ACCOUNT_ID}"
}
},
"airtable": {
"type": "stdio",
"command": "npx",
"args": ["-y", "@airtable/mcp-server"],
"env": {
"AIRTABLE_API_KEY": "${AIRTABLE_API_KEY}",
"AIRTABLE_BASE_ID": "${AIRTABLE_BASE_ID}"
}
},
"google-sheets": {
"type": "stdio",
```

```
"command": "npx",
"args": ["-y", "@anthropic/google-sheets-mcp"],
"env": {
"GOOGLE_CLIENT_ID": "${GOOGLE_CLIENT_ID}",
"GOOGLE_CLIENT_SECRET": "${GOOGLE_CLIENT_SECRET}"
}
},
"analytics": {
"type": "stdio",
"command": "node",
"args": ["mcp-servers/analytics/index.js"],
"env": {
"GA4_PROPERTY_ID": "${GA4_PROPERTY_ID}",
"GA4_CREDENTIALS_PATH": "${GA4_CREDENTIALS_PATH}"
}
},
"filesystem": {
"type": "stdio",
"command": "npx",
"args": ["-y", "@anthropic/filesystem-mcp", "./contexts", "./templates"]
}
}
}
```

MCP Server Reference

MCP Server	Transport	Source	Tools Exposed	Used By
vwo	stdio	Custom (mcp-servers/vwo/)	create_experiment, get_results, pause_test, get_live_data, list_experiments	Experiment Designer, Monitor, Analyst
airtable	stdio	npm: @airtable/mcp-server	list_records, create_record, update_record, search_records	All modules (experiment tracking DB)
google-sheets	stdio	npm: @anthropic/google-sheets-mcp	read_sheet, write_sheet, create_sheet	Report Generator, Dashboard
analytics	stdio	Custom (mcp-servers/analytics/)	get_metrics, get_segments, get_funnel, get_retention	Analysis Engine, Monitor, Dashboard

MCP Server	Transport	Source	Tools Exposed	Used By
filesystem	stdio	npm: @anthropic/filesystem-mcp	read_file, write_file, list_directory	All modules (context & template access)

Environment Variables Setup

MCP servers use environment variables for secrets. Create a .env file locally (gitignored) and set variables in your shell profile:

```
# Add to ~/.bashrc or ~/.zshrc
export VWO_API_TOKEN="your-vwo-api-token"
export VWO_ACCOUNT_ID="your-vwo-account-id"
export AIRTABLE_API_KEY="your-airtable-api-key"
export AIRTABLE_BASE_ID="your-airtable-base-id"
export GA4_PROPERTY_ID="your-ga4-property-id"
export GA4_CREDENTIALS_PATH="path/to/ga4-credentials.json"
export GOOGLE_CLIENT_ID="your-google-client-id"
export GOOGLE_CLIENT_SECRET="your-google-client-secret"
```

Security Best Practice

Never commit API keys or secrets to Git. The .mcp.json file uses ${ENV_VAR} syntax which Claude Code expands at runtime from your environment.

Add .env to your .gitignore immediately. Share secrets with team members through a secure channel like 1Password or Vault.

Settings, Hooks & Permissions

Project Settings (.claude/settings.json)

This file controls shared project settings. It is committed to Git so the team has consistent behavior.

```
{
"permissions": {
"allow": [
"Read",
"Write",
"Edit",
"Bash(npm:*)",
"Bash(node:*)",
"Bash(python3:*)",
"mcp__airtable__*",
"mcp__vwo__get_*",
"mcp__analytics__*",
"mcp__filesystem__*",
"mcp__google-sheets__read_*"
],
"deny": [
"Bash(rm -rf:*)",
"Bash(git push --force:*)"
]
}
}
```

Local Settings (.claude/settings.local.json)

This file is gitignored and stores personal preferences. Each team member has their own.

```
{
"permissions": {
"allow": [
"mcp__vwo__create_*",
"mcp__vwo__pause_*",
"mcp__airtable__create_*",
"mcp__google-sheets__write_*"
]
},
"hooks": {
"PostToolUse": [
{
"matcher": "mcp__vwo__create_experiment",
```

```
"hooks": [
{
"type": "command",
"command": "echo 'Experiment created' >> xos-audit.log"
}
]
}
]
}
}
```

Understanding Hooks

Hooks are lifecycle callbacks that execute at specific points during Claude Code's operation. They are powerful for adding audit logging, validation, and automation to xOS Brain.

Hook Event	When It Fires	xOS Brain Use Case
PreToolUse	Before Claude executes any tool	Validate experiment parameters before VWO API calls
PostToolUse	After a tool completes successfully	Log all MCP operations to an audit trail
Stop	When Claude finishes a response	Auto-save conversation summaries to experiment records
SessionStart	When a new Claude Code session begins	Load the active client context automatically

> **Permission Strategy: Read-First, Write-Later**
>
> Start with settings.json allowing only READ operations on MCPs (get_*, read_*, list_*). This is safe for the whole team.
>
> Each team member can then opt-in to WRITE operations (create_*, update_*, pause_*) in their settings.local.json.
>
> This ensures nobody accidentally creates or modifies experiments without explicitly granting themselves permission.

Context Layer — Client-Specific Data

The Context Layer is what makes xOS Brain **reusable across clients**. Instead of hardcoding business rules, KPIs, and domain knowledge into skills, you store them in the **contexts/** directory. Each client gets their own folder with a standardized structure.

Context Folder Structure

```
contexts/
■■■ _template/ # Copy this to create new clients
■ ■■■ CLAUDE.md # Client-specific Claude instructions
■ ■■■ config.yaml # Client configuration
■ ■■■ brand-voice.md # Brand voice & terminology
■ ■■■ knowledge/
■ ■■■ past-experiments.md # Historical experiment data
■ ■■■ business-model.md # Business model canvas
■ ■■■ constraints.md # Known constraints
■
■■■ acme-corp/ # Real client example
■■■ CLAUDE.md
■■■ config.yaml
■■■ brand-voice.md
■■■ knowledge/
■■■ past-experiments.md
■■■ business-model.md
■■■ constraints.md
```

Client config.yaml Template

```
# Client Configuration
client:
name: "ACME Corporation"
industry: "E-commerce"
website: "https://acme.com"

# Key Performance Indicators
kpis:
primary:
- metric: conversion_rate
current: 3.2%
target: 4.0%
- metric: average_order_value
current: $67
target: $75
secondary:
```

```
- metric: bounce_rate
current: 45%
target: 38%

# Active OKRs this quarter
okrs:
- objective: "Increase checkout conversion"
key_results:
- "Reduce cart abandonment by 15%"
- "Increase payment completion rate to 85%"

# Tools & Platforms
tools:
ab_testing: "VWO"
analytics: "GA4"
experiment_tracker: "Airtable"
reporting: "Google Sheets"

# Constraints
constraints:
min_sample_size: 1000
max_test_duration_days: 28
approval_required_for: ["pricing", "checkout", "homepage"]
```

Client CLAUDE.md Template

Each client's CLAUDE.md provides context-specific instructions that override or extend the project-level CLAUDE.md.

```
# ACME Corporation - xOS Context

## Business Context
ACME is a B2C e-commerce platform selling consumer electronics.
Primary revenue model: direct online sales with average order value of $67.
Peak traffic: Q4 (Black Friday through Christmas).

## Current Focus
Q1 2026 focus: Checkout conversion optimization.
Do NOT test on: mobile app (separate team), email campaigns (marketing owns).

## Terminology
- "PDP" = Product Detail Page
- "Add to Bag" = Their CTA (not "Add to Cart")
- "Express Checkout" = One-click purchase flow

## Guardrails
```

```
- Never modify pricing without VP approval (flag to user)
- Holiday freeze: No new tests Dec 15 - Jan 5
- Minimum test duration: 14 days (due to weekly cycles)
```

Step-by-Step Build Order

Now that you understand every component, here is the **exact sequence** to build xOS Brain from scratch. Follow these steps in order. Each step builds on the previous one.

> **Before You Start**
>
> Prerequisites: Node.js 18+, Claude Code CLI installed (claude command available), GitHub CLI (gh), a GitHub account.
>
> Time estimate: ~4-6 hours for the initial setup. Skills and agents can be refined incrementally.

Phase 1: Repository Foundation (30 minutes)

Set up the GitHub repository and core file structure.

```
# Step 1: Clone your repo and enter it
git clone https://github.com/bjarnbrunenberg-cloud/xOS-Brain.git
cd xOS-Brain

# Step 2: Create the folder structure
mkdir -p .claude/skills
mkdir -p .claude/agents
mkdir -p .claude/agent-memory/experiment-orchestrator
mkdir -p .claude/agent-memory/data-analyst
mkdir -p .claude/agent-memory/monitor-agent
mkdir -p contexts/_template/knowledge
mkdir -p templates/hypotheses
mkdir -p templates/experiments
mkdir -p templates/reports
mkdir -p mcp-servers/vwo
mkdir -p mcp-servers/airtable
mkdir -p mcp-servers/analytics
mkdir -p docs

# Step 3: Initialize Node.js project
npm init -y

# Step 4: Create .gitignore
cat > .gitignore << 'EOF'
node_modules/
.env
.claude/settings.local.json
*.log
.DS_Store
EOF
```

Phase 2: CLAUDE.md & Settings (30 minutes)

Create the master memory file and project settings.

- Create CLAUDE.md at the repository root using the complete template from Module 2 of this guide.
- Create .claude/settings.json with the permission rules from Module 7 (read-only MCP access by default).
- Create .claude/settings.local.json with your personal write permissions for MCP tools.
- Test by running: claude and verifying Claude acknowledges the xOS Brain identity.

Phase 3: Skills — Build the 9 Modules (2-3 hours)

Build each skill file one at a time. Start with the most fundamental modules and work outward.

Order	Skill	Priority	Rationale
1	hypothesis-factory.md	Critical	Foundation: everything starts with a hypothesis
2	experiment-designer.md	Critical	Transforms hypotheses into actionable test plans
3	analysis-engine.md	Critical	Interprets results — core value delivery
4	strategic-alignment.md	High	Ensures tests align with business goals
5	learning-repository.md	High	Builds the knowledge base over time
6	report-generator.md	Medium	Automates report creation
7	experiment-monitor.md	Medium	Tracks running experiments
8	next-best-test.md	Medium	Requires data from previous experiments
9	dashboard.md	Lower	Aggregation view — build after other modules work

For each skill, follow this process:

- Copy the skill file structure template from Module 4.

- Write the description (trigger condition) — be specific about keywords and user intents.
- Write the detailed instructions — include step-by-step workflow, output format, and guardrails.
- Test the skill by starting a Claude Code session and typing a matching request.
- Iterate on the description if the skill doesn't activate when expected, or activates too often.

Phase 4: MCP Configuration (1-2 hours)

Set up the external tool connections.

Order	MCP Server	Setup Steps
1	**filesystem**	Add to .mcp.json — no keys needed. Points to contexts/ and templates/ directories.
2	**airtable**	Create Airtable base with Experiments and Learnings tables. Get API key. Add to .mcp.json.
3	**google-sheets**	Set up Google Cloud OAuth credentials. Add to .mcp.json.
4	**vwo**	Build custom MCP server in mcp-servers/vwo/. Get VWO API token. Add to .mcp.json.
5	**analytics**	Build custom MCP server in mcp-servers/analytics/. Set up GA4 service account.

How to add an MCP server via CLI:

```
# Add a community MCP server
claude mcp add airtable --scope project -- npx -y @airtable/mcp-server

# Add a custom MCP server
claude mcp add vwo --scope project -- node mcp-servers/vwo/index.js

# Verify all MCP servers are configured
claude mcp list
```

Phase 5: Agents (1 hour)

Create the agent definitions after skills and MCPs are working.

- Create experiment-orchestrator.md in .claude/agents/ using the template from Module 5. This is the flagship agent.

- Create data-analyst.md — focused on statistical analysis with access to analytics and VWO results.
- Create report-writer.md — focused on generating professional experiment reports.
- Create monitor-agent.md — lightweight agent (haiku model) for checking running experiment health.
- Test each agent by dispatching it from a Claude Code session.

Phase 6: Context Layer Setup (30 minutes)

- Finalize the _template/ folder with all template files (CLAUDE.md, config.yaml, brand-voice.md, knowledge/ files).
- Create your first real client context by copying _template/ to a named folder (e.g., contexts/acme-corp/).
- Customize the client's config.yaml with their real KPIs, OKRs, and tool preferences.
- Write the client's CLAUDE.md with their specific business context and guardrails.
- Test by asking Claude about the client's business goals and verifying it pulls from the context.

Phase 7: Integration Testing (1 hour)

Run through a complete experiment lifecycle to verify everything works together.

Integration Test Script

1. Start Claude Code in the xOS-Brain directory
2. Say: "I want to improve checkout conversion for ACME Corp"
3. Verify: Strategic alignment skill activates, reads ACME context
4. Say: "Generate hypotheses for this"
5. Verify: Hypothesis Factory activates, produces ICE-scored hypotheses
6. Say: "Design an experiment for hypothesis #1"
7. Verify: Experiment Designer activates, calculates sample size, produces plan
8. Say: "Run the full experiment orchestrator on this"
9. Verify: Orchestrator agent launches, follows the multi-step workflow
10. Check: Airtable has a new experiment record, report is generated

Airtable Schema — Experiment Tracking Database

Airtable serves as the **central experiment database** for xOS Brain. Here is the recommended schema for your Airtable base.

Experiments Table

Field	Type	Description
experiment_id	Auto Number	Unique identifier
hypothesis	Long Text	The IF/THEN/BECAUSE hypothesis statement
status	Single Select	Options: Draft, Designed, Running, Paused, Completed, Cancelled
test_type	Single Select	A/B Test, Multivariate, Fake Door, Survey, etc.
primary_metric	Single Line	The primary metric being measured
ice_score	Number	ICE priority score (1-10)
start_date	Date	When the experiment launched
end_date	Date	When the experiment concluded
sample_size_target	Number	Required sample size
sample_size_actual	Number	Actual sample reached
result	Single Select	Win, Loss, Inconclusive, Stopped Early
confidence_level	Percent	Statistical confidence achieved
lift	Percent	Measured lift on primary metric
client	Single Line	Client context name
owner	Collaborator	Team member responsible
learnings	Long Text	Key takeaways and insights
linked_report	URL	Link to generated report

Learnings Table

Field	Type	Description
learning_id	Auto Number	Unique identifier
experiment_id	Linked Record	Links to Experiments table
insight	Long Text	The learning or insight discovered
category	Single Select	Conversion, Retention, Pricing, UX, Content, Other
confidence	Single Select	High, Medium, Low
date_discovered	Date	When this learning was recorded
applied_to	Long Text	Which future experiments used this learning
tags	Multiple Select	Searchable tags for pattern detection

Prioritized Roadmap — What to Build When

You don't need to build everything at once. Follow this **phased approach** to get xOS Brain operational as fast as possible, then iterate.

Phase	Timeline	What You Build	Outcome
1. Foundation	Days 1-2	CLAUDE.md, folder structure, .gitignore, settings.json	Claude Code recognizes xOS Brain identity
2. Core Skills	Days 3-5	hypothesis-factory, experiment-designer, analysis-engine skills	Can generate hypotheses, design tests, analyze results
3. First MCP	Days 6-8	Airtable MCP + Experiments table schema	Experiment tracking database is connected
4. Orchestrator	Days 9-10	experiment-orchestrator agent + data-analyst agent	End-to-end experiment automation works
5. Context Layer	Days 11-12	Client template + first real client context	xOS Brain is client-aware and reusable
6. Expand MCPs	Days 13-16	VWO MCP, Analytics MCP, Google Sheets MCP	Full tool integration for live experiments
7. Full Suite	Days 17-20	Remaining skills (5-9), remaining agents, templates	Complete xOS Brain operational
8. Polish	Days 21-24	Integration testing, hooks, permission tuning, documentation	Production-ready xOS Brain

Quick Win: Start with Phase 1-2 Today

You can have the folder structure, CLAUDE.md, and your first three skills working within a single afternoon.

Even without MCPs connected, the skills provide massive value: structured hypothesis generation, experiment design frameworks, and statistical analysis guidance.

Add MCPs incrementally as you get API access to each platform.

Appendix — Quick Reference Commands

Essential Claude Code Commands

Command	Purpose
claude	Start a Claude Code session in the current directory
claude mcp list	List all configured MCP servers
claude mcp add <name> --scope project	Add an MCP server to the project scope
claude mcp remove <name>	Remove an MCP server
/agent experiment-orchestrator	Dispatch the experiment orchestrator agent
/agent data-analyst	Dispatch the data analyst agent
/memory	View and edit CLAUDE.md memory files

Useful Git Workflow

```
# After making changes to skills or agents
git add .claude/skills/ .claude/agents/ CLAUDE.md
git commit -m "feat: add hypothesis-factory skill"
git push origin main

# Create a new client context
cp -r contexts/_template contexts/new-client
git add contexts/new-client/
git commit -m "feat: add new-client context"
```

File Naming Conventions

File Type	Convention	Example
Skills	kebab-case.md	hypothesis-factory.md, experiment-designer.md
Agents	kebab-case.md	experiment-orchestrator.md, data-analyst.md
Client contexts	kebab-case/	acme-corp/, startup-xyz/
Templates	kebab-case.md	ab-test.md, conversion-hypothesis.md
MCP servers	kebab-case/	vwo/, google-sheets/

This implementation guide gives you everything you need to build the xOS Experimentation Brain from the ground up. Start with Phase 1, iterate through each phase, and you will have a fully operational experimentation OS within 3-4 weeks. The key is to **start building today** — even a partial xOS Brain with just CLAUDE.md and three core skills delivers immediate value.